

CSCE 606 SOFTWARE ENGINEERING · MODULE 06 · LECTURE NOTES

Test Design

Choosing which cases to write.

MODULE OUTCOMES

- Use equivalence partitioning and boundary analysis.
- Write property-based tests and choose good properties.
- Use test doubles appropriately and know their costs.
- Test error paths and concurrency deliberately.
- Write characterisation tests for code you do not understand.

1 Choosing cases

The input space of any non-trivial function is effectively infinite. Test design is the problem of choosing a small finite subset that finds most of the bugs.

1.1 Equivalence partitioning

Divide the input space into classes within which you expect the code to take the same path and behave the same way. Test one representative from each; a second representative from the same class exercises the same logic and adds little.

```
withdraw(balance, amount):  
  
    amount < 0           -- invalid  
    amount == 0         -- boundary, possibly special  
    0 < amount < balance -- the ordinary case  
    amount == balance   -- boundary, empties the account  
    amount > balance    -- insufficient funds
```

Five well-chosen tests cover the meaningful behaviour far better than five hundred random values, all but a handful of which land in the same class. Note that partitioning is a **hypothesis about the implementation** — which is why it is effective and why it can miss a class you did not anticipate.

BUGS CLUSTER AT BOUNDARIES

Off-by-one errors, incorrect comparison operators ($<$ instead of $\leq$), overflow, and empty-case handling all occur at the *edges* of equivalence classes rather than in their interiors.

So for every boundary, test three values: just below, exactly at, and just above.

The standard boundaries are worth memorising: empty collection, single element, many elements; zero, one, maximum; INT_MAX and INT_MIN; empty string, one character, maximum length; first element, last element; null or absent.

This is where to concentrate the test budget. A value drawn from the middle of a class almost never finds a defect that the class's boundary values would not have found.

2 Property-based testing

Example-based tests check the cases you thought of. Property-based tests state a claim that should hold for *all* inputs, and let a framework search for counterexamples.

```
@given(lists(integers()))
def test_sort_is_ordered(xs):
    r = sort(xs)
    assert all(r[i] <= r[i+1] for i in range(len(r) - 1))

@given(lists(integers()))
def test_sort_is_a_permutation(xs):
    assert Counter(sort(xs)) == Counter(xs)
```

The second property is not optional: without it, an implementation that simply returns the empty list satisfies the first. Properties usually come in pairs — one constraining the output's shape, one constraining its relationship to the input.

SHRINKING IS WHAT MAKES THIS PRACTICAL

When a property fails, the framework does not report the 200-element random list that triggered it. It automatically **shrinks** the counterexample, repeatedly simplifying while the failure persists, and reports something minimal — often two or three elements.

A minimal counterexample is usually diagnostic on sight: `[0, -0.0]` or `['', '']` tells you immediately what you forgot.

Without shrinking, property testing would produce failures too large to interpret. With it, the framework finds the edge case *and* explains it.

Property pattern	Form	Finds
Round trip	$\text{decode}(\text{encode}(x)) == x$	Serialisation, parsing, and encoding bugs. The single most productive pattern — these bugs are almost invisible to example tests.
Invariant	Something true of every output.	Output sorted, tree balanced, total non-negative, sum preserved.
Idempotence	$f(f(x)) == f(x)$	Normalisation, sorting, absolute value, deduplication, abs, set.
Oracle	Compare against a slow, obviously-correct implementation.	Optimised code. Test the fast version against the naive one on random inputs.
Metamorphic	A known relation between two outputs.	$\text{sort}(xs + ys)$ contains exactly the elements of both; $f(2x) == 2f(x)$ for a linear function.

3 Test doubles

Kind	Behaviour	Appropriate when
Dummy	Satisfies a parameter; never actually used.	The argument is irrelevant to the behaviour under test.
Stub	Returns pre-arranged answers.	You need the dependency to produce a particular input.
Spy	A stub that records the calls it received, for later inspection.	The call itself is the observable effect — 'did it send the email?'
Mock	Pre-programmed with expectations; fails if they are not met.	Sparingly. See below.
Fake	A genuine, working, simplified implementation.	Usually the best choice. An in-memory repository, a map-backed cache.

PREFER FAKES TO MOCKS

A **mock** asserts that particular calls were made with particular arguments. That couples the test to the implementation (Module 05), so refactoring breaks it. Worse, the mock encodes *your belief* about how the dependency behaves, and if that belief is wrong, the test passes while the system is broken.

A **fake** is a working implementation with simplified internals — an in-memory version of a repository, a simple in-process queue. Tests against it exercise real behaviour, are readable, and survive restructuring.

Better still, use the real thing where it is fast enough. An in-memory SQLite database is a real SQL database and runs in milliseconds. The best test double is often no double.

One caveat: a fake whose behaviour diverges from the real implementation provides false confidence. Ideally the fake is written and maintained by whoever owns the real implementation, and is covered by a shared contract test suite that both must pass.

4 The hard cases

4.1 Error paths

THE LEAST-EXECUTED CODE IS THE MOST DANGEROUS

Error handling is simultaneously the code most likely to contain defects — because it is rarely executed and rarely reviewed with care — and the code that runs precisely when the system is already in trouble.

A retry path that has never been executed is a retry path that does not work. A fallback that has never been exercised will fail on the day it is needed, and will do so while the primary path is also failing.

So force the errors. Inject failures at dependency boundaries, fill the disk, sever the network, exhaust the connection pool, return malformed and truncated responses, return success after a timeout has already fired.

Fault injection at the unit level and **chaos engineering** at the system level are the same idea at different scales: make the failure happen when you are watching, rather than at three in the morning.

4.2 Concurrency

TESTING CANNOT ESTABLISH CONCURRENT CORRECTNESS

A race condition manifests only under a particular interleaving. Running a test ten thousand times may never produce that interleaving, and production may produce it within an hour (CSCE 611 Module 04).

Testing is therefore necessary and **not sufficient** for concurrent code, and this should be stated plainly rather than papered over.

What helps: ThreadSanitizer and similar tools, which reason about happens-before relationships rather than observed outcomes and therefore find races on executions where nothing went wrong; stress testing with randomised scheduling and injected delays; deterministic simulation, where the test controls the interleaving explicitly and can replay a failure.

What helps most is reducing sharing. Code with no shared mutable state has no races to find, and that is a design decision rather than a testing one.

4.3 Characterisation tests

The common situation: you must modify code that has no tests, that you did not write, and whose correct behaviour nobody can state.

- 1 Write tests that capture what the code **currently does** — not what it should do. Run it, observe the output, assert that output.
- 2 Where the output is complex, assert on a serialisation or a hash of it.
- 3 Build up enough of these to cover the paths your change will touch.
- 4 Now refactor. The characterisation tests tell you when behaviour changed.
- 5 **Afterwards**, examine the captured behaviour for bugs — there will be some — and fix them deliberately, one at a time, updating the corresponding test with a comment explaining why the expected value changed.

These are explicitly not correctness tests. They are a safety net that converts 'I hope I did not break anything' into 'I know exactly what I changed'. Michael Feathers's *Working Effectively with Legacy Code*

develops the technique, and it is the only workable approach to untested code that must be changed.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.031 — Testing (partitioning and boundary analysis)	The most systematic free treatment of choosing test cases.
Hypothesis documentation — 'What is property-based testing?'	Excellent explanation plus a usable library. The equivalents in other languages (QuickCheck, fast-check, proptest) share the ideas.
Martin Fowler — Mocks Aren't Stubs	The taxonomy of doubles and the argument about classical versus mockist testing. Still the clearest statement of both positions.
Michael Feathers — Working Effectively with Legacy Code	Characterisation tests and seam-finding. The standard reference for testing code that was not designed to be tested.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a function with at least three parameters and derive its equivalence classes. Write one test per class, then add boundary tests. Compare the defect-finding power against twenty random inputs.
- 2 Write property-based tests for a sorting function, including both the ordering and the permutation property. Then deliberately implement sort as return [] and confirm which property catches it.
- 3 Find a round-trip property in your own code — serialisation, parsing, encoding — and test it with generated inputs. Report whatever it finds.
- 4 Write an oracle test: implement something the slow obvious way and the fast way, and compare them on random inputs.
- 5 Replace a mock-heavy test with one using a fake. Then refactor the code under test and confirm the fake-based test survives while the mock-based one would not.
- 6 Write a fault-injection test for an error path in your code that has never been executed. Report what you found.
- 7 Take an undocumented function you did not write and build a characterisation test suite for it. Then refactor it and report whether any test caught a behaviour change.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is equivalence partitioning, and why does a second value from the same class add little?
- 2 Why do bugs cluster at boundaries, and which three values do you test for each?
- 3 What does a property-based test assert that an example-based test does not, and why is shrinking essential?
- 4 Name five property patterns and give an example of each.
- 5 Distinguish a mock from a fake, and give two reasons to prefer the fake.
- 6 Why are error paths both the most dangerous and the least tested code?
- 7 Why can testing not establish the correctness of concurrent code, and what helps instead?
- 8 What is a characterisation test and what is it for?